

Submission Worksheet

IT202-450-M2026 / it202-milestone-2-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 2 - 2026
Student	Michos C. (mrc82)
Status	submitted
Worksheet Progress	100%
Potential Grade	10.00/10.00 (100.00%)
Received Grade	0.00/10.00 (0.00%)
Updated	8/5/2026, 10:27:39 PM

Grading Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-2-2026/grading/mrc82>

View Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-2-2026/mrc82>

Instructions

Verify that your lesson-built project data system is complete and functional, then collect focused evidence for the database, server-side API import, manual creation, admin list, public list, public details, edit, and delete requirements. Most coding should already be complete from the lessons; this worksheet focuses on confirming the full data flow, fixing remaining issues, and explaining your decisions.

Assignment Setup

- Required branch: `Milestone2`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required deployed evidence target: working `qa` site before production promotion
- Recommended order:
 - Read the whole worksheet.
 - Review the modern project proposal and Milestone 2 assignment requirements.
 - Check out `qa`, pull the latest version, and create the milestone branch.
 - Ensure the related API, database-helper, and CRUD lessons are complete in your project.
 - Test each data source and CRUD behavior.
 - Merge the milestone branch into `qa`, test the deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the milestone branch with `git checkout -b Milestone2`.
4. Commit focused verification and fixes in small pieces. If there is nothing to commit, wait until you add the PDF to your repository.
5. Push `Milestone2`.
6. Open a pull request from `Milestone2` into `qa` and keep it open while you work

your repository.

5. Push `Milestone2`.
6. Open a pull request from `Milestone2` into `qa` and keep it open while you work.
7. After all Milestone 2 requirements pass locally, merge the pull request into `qa`.
8. Gather evidence from the deployed `qa` site and fill out the worksheet.
9. Export the worksheet PDF from the Learn Courses Platform.
10. Save the PDF in the repository under `docs/milestone02/`.
11. Add, commit, and push the PDF to the `Milestone2` branch.
 - `git add .`
 - `git commit -m "Add milestone 2 worksheet PDF"`
 - `git push origin Milestone2`
12. Upload the same PDF to Canvas.
13. Create and merge a pull request from `qa` into `prod` when production evidence is required.
14. Switch back to `qa`.
15. Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 2 feature:

- Add a comment with your UCID, date, and a brief summary of each feature area.
- Load shared setup through `lib/app.php` and follow the course API, database, validation, flash, session, and role-check patterns.
- Map API data in server-side PHP, keep secrets in the supported environment configuration, and store at least three useful API-derived values in real table columns.
- Keep API-imported and manual records compatible in the same table shape when possible, clearly identify their source, and apply one duplicate-record rule consistently.
- Include `id`, `created`, and `modified` on project data tables with reasonable types and constraints.
- Validate form writes in HTML, the page's `validate()` function, and PHP; retain safe sticky values after an error.
- Use prepared statements or shared database helpers, friendly flash messages, and `error_log()` for technical errors.
- Enforce the authorized role on API import, manual creation, edit, and delete pages; hide management actions from users who cannot use them.
- Keep navigation and styling consistent with the supported roles.
- Test the supported deployed `qa` URLs before collecting evidence.
- Use your own API, mapped data, database behavior, code, branch, pull request, URLs, and explanations.

Evidence Format

Most sections use the same evidence pattern:

- Images:
 - Show relevant code and matching browser or database output.
 - Make sure UCID/date comments are visible in code evidence.
 - Make sure the deployed `qa` URL is visible in browser evidence.
 - One combined screenshot is acceptable when the code and output are both readable.
 - Reuse earlier evidence when it already proves the same requirement.
- URLs:
 - Provide direct GitHub file links from the milestone branch.
 - Provide anticipated production URLs for deployed PHP pages.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - PHP page URLs must end in `.php`; zero credit for that URL entry if it does not.
- Response:
 - Explain the logic in your own words.
 - Describe the decisions made by your PHP, SQL, JavaScript, API mapping, or database design.

- Response:
 - Explain the logic in your own words.
 - Describe the decisions made by your PHP, SQL, JavaScript, API mapping, or database design.
 - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the `Milestone2` branch and pull request into `qa` have been pushed.
- Confirm the deployed `qa` site was reviewed before collecting evidence.
- Test API import, manual creation, admin list, public list, public details, edit, and delete behavior.
- Test relevant filtering and sorting, the server-side limit rule, and friendly failure handling.
- Confirm anticipated production URLs use the tested `qa` paths with `-qa` changed to `-prod`.
- Commit the worksheet PDF under `docs/milestone02/` and upload the same PDF to Canvas.

Submission Progress

100%

Section #1: (2 pts.) Project Data Design

100%

- Goal: show that the database stores useful mapped API data and compatible manual records.
- Expected behavior:
 - Project tables include `id`, `created`, and `modified`.
 - At least three useful API-derived values use real columns with reasonable types and constraints.
 - API and manual records share the same table shape whenever possible and can be distinguished.
 - Nested or related API data is transformed into columns or related tables where appropriate.

Task #1 (2 pts.) – Project Data Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant table structure and several rows in the VS Code database extension or equivalent database view.
- Include `id`, `created`, `modified`, the API/manual indicator, at least three mapped project fields, one API record, and one manual record.



Characters table structure with mapped fields, API/manual records, source flag, and timestamps.

Part 2: URLs 100%

Details

- Paste direct GitHub links for the relevant SQL or migration file(s) and API mapping/wrapper file.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/sql/007_create_characters.sql 🍏

URL #2

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/sql/007_create_characters.sql 🍏

URL #2

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/lib/project_api.php 🍏

URL #3

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/lib/character_helpers.php 🍏

🔗 Part 3: Response 100%

① Details

- Explain how the API response became the table design, how API and manual rows are distinguished, and one important type, constraint, index, or relationship decision.

Response

So the API data is mapped into separate database columns such as name, status, species, gender, origin, location, and image URL. API records use `is_api = 1` and include an `api_id`, while manual records use `is_api = 0` and leave `api_id` empty, allowing both types to share the same table. The table also uses an unsigned primary ID, reasonable text limits, and created and modified timestamps to keep the data organized and track changes.

Supports **bold**, *italic*, [links](#)(url), ``code``, etc.

433 chars

Section #2: (2 pts.) Admin Api Import And Manual Create

100%

- Goal: show that an authorized admin can import API records and create compatible manual records safely.
- Expected behavior:
 - Server-side PHP requests, validates, and maps API data before database writes.
 - Manual creation uses HTML, JavaScript, and PHP validation.
 - Duplicate API records follow one predictable rule.
 - Server-side role checks protect both workflows.

📁 Task #1 (2 pts.) – API Import And Manual Create Evidence

100%

📁 Combo #1

🖼️ Part 1: Images 100%

① Details

- Show deployed **qa** evidence for one successful API import and one successful manual creation.
- Show one friendly validation, duplicate, or API failure message.
- Include focused code evidence for the mapping/write path and admin guard. Do not expose secrets.



Deployed QA manual character creation with a success message.



Deployed QA API import showing successful Morty import.

message.



Duplicate Morty API import rejected with a friendly message.



Admin authorization and server-side API mapping with prepared database logic.

Part 2: URLs 100%

Details

- Paste the external API documentation URL.
- Paste direct GitHub links for the API wrapper/import logic and manual creation page.
- Paste anticipated production URLs for the page(s) used to import and create records.

URL #1

<https://rickandmortyapi.com/documentation> 🍏

URL #2

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/lib/project_api.php 🍏

URL #3

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/lib/character_helpers.php 🍏

URL #4

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone2/public_html/project/admin/create_character.php 🍏

URL #5

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/create_character.php 🍏

Part 3: Response 100%

Details

- Trace one API record from request to mapped database row, explain how manual creation uses the same data shape, and describe the duplicate strategy.

Response

Basically the admin search sends the character name to the API wrapper, which requests the Rick and Morty API from server-side PHP and maps the returned values into the Characters table. API records store the external api_id and use is_api = 1, while manual records use the same columns but leave api_id empty and use is_api = 0. Manual entries are validated in HTML, JavaScript, and PHP before a prepared database write. Before importing, the system checks the API ID and rejects an existing character with a friendly duplicate message.

Supports **bold**, *italic*, [links](url), `code`, etc.

537 chars

Section #3: (1 pt.) Admin Data List

100%

- Goal: show that an authorized admin can manage useful summaries of API and manual records.

100%

- Goal: show that an authorized admin can manage useful summaries of API and manual records.
- Expected behavior:
 - The page is protected by the authorized role.
 - API-imported and manual records appear together as useful summaries.
 - The list supports project-relevant filtering and sorting plus a server-validated limit from 1 through 100.
 - Invalid limits default to 10, selected controls remain sticky, and empty results show friendly feedback.
 - Each row links to the public detail page and includes the allowed edit and delete controls.

Task #1 (1 pt.) – Admin List Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the deployed **qa** list as an authorized admin.
- Include API and manual records, filter, sort, limit, detail-link, edit, and delete controls.
- Show one useful filtered or sorted result and one friendly no-results state.
- Include focused code evidence for the list query, limit validation, role guard, and management controls.



Deployed QA Admin list showing API and manual records with filter, sort, limit, edit, and delete controls.



Deployed QA Admin list filtered to the Morty Smith API record with sticky controls and management actions.



Deployed QA Admin list showing a friendly no-results message with the selected filter retained.



Admin role protection with validated filtering, sorting, and limit logic.



Prepared filtered list query with enforced sorting and limit, plus edit and confirmed delete controls.

Part 2: URLs 100%

Details

- Paste a direct GitHub link for the admin list page.
- Paste the anticipated production URL for the admin list page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/admin/list_characters.php 🍏

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/admin/list_characters.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/list_characters.php 🍏

Part 3: Response 100%

Details

- Explain how filters, sorting, and the limit change the query and why the page requires the management role.

Response

The selected filters add matching conditions to the database query, while the sort option chooses from approved column orders. The result limit is validated from 1 through 100 and defaults to 10 when invalid. This page requires the Admin role because it includes management controls that can edit and permanently delete character records.

Supports **bold**, *italic*, [links](url), ``code``, etc.

338 chars

Section #4: (1 pt.) Public Data List

100%

- Goal: show that non-admin users can browse useful record summaries without receiving management access.
- Expected behavior:
 - API-imported and manual records appear together with a link to the public detail page.
 - The page supports project-relevant filtering and sorting plus a server-validated limit from 1 through 100.
 - Invalid limits default to 10, selected controls remain sticky, and empty results show friendly feedback.
 - Non-admin users do not see edit or delete controls.
 - A shared role-aware list page is acceptable when it produces the same public and admin behavior.

Task #1 (1 pt.) – Public List Evidence

100%

Combo #1

Part 1: Images 100%

Details

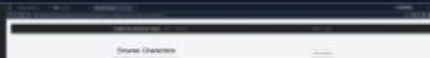
- Show the deployed **qa** list as a non-admin user.
- Include API and manual records, filter, sort, limit, and detail-link controls.
- Show one useful filtered or sorted result and one friendly no-results state.
- Include focused code evidence for the list query, limit validation, and conditional absence of management controls.



Deployed QA public list showing API and manual records with filtering, sorting, limits, and detail links without management cont



Deployed QA public list filtered to Morty Smith with sticky controls and public detail access.





Deployed QA public list showing a friendly no-results message with the selected filter retained.



Public list query with filtering, sorting, and limits, plus View-only access without Admin controls.

Part 2: URLs 100%

Details

- Paste a direct GitHub link for the public list page.
- Paste the anticipated production URL for the public list page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/characters.php 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/characters.php> 🍏

Part 3: Response 100%

Details

- Explain how the public list reuses or differs from the admin list and how it prevents non-admin management access.

Response

The public list reuses the same filtered, sorted, and validated database query as the Admin list, including the 1–100 result limit and default of 10. However, it provides only View links and does not include Edit or Delete actions. This allows non-admin users to browse API and manual records without receiving management access.

Supports **bold**, *italic*, [links](url), `code`, etc.

329 chars

Section #5: (1 pt.) Public Record View

100%

- Goal: show a public detail page that safely displays one project record.
- Expected behavior:
 - The page validates the requested ID and shows more information than the list summary.
 - A missing or invalid ID redirects with a friendly message.
 - Non-admin users can view the detail without edit or delete controls.
 - When the same page offers management actions to an authorized role, those actions appear only for that role.

Task #1 (1 pt.) – Public Record View Evidence

100%

Combo #1

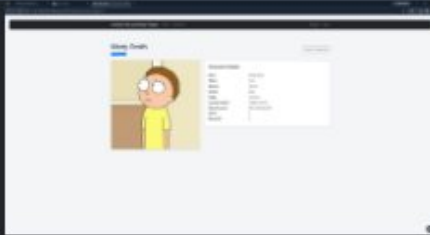
Part 1: Images 100%

Details

Show one deployed detail page as a non-admin user with information not shown in the list

Details

- Show one deployed **qa** detail page as a non-admin user with information not shown in the list.
- Show one friendly invalid-ID state.
- Include focused code evidence for ID validation, the single-record query, and the conditional management-action check.



Deployed QA public detail page showing expanded Morty Smith information without management controls.



Invalid public character ID redirects safely with a friendly record-not-found message.



Validated record ID, single-record query, friendly missing-record redirect, and conditional Admin-only access.

Part 2: URLs 100%

Details

- Paste a direct GitHub link for the detail page.
- Paste the anticipated production URL for one valid detail-page example.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/character.php 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/character.php?id=5> 🍏

Part 3: Response 100%

Details

- Explain how the page validates the requested record, why it shows more detail than the list, and how it keeps management actions role-specific.

Response

The detail page validates the requested character ID before running a prepared single-record query. Missing or invalid records redirect safely with a friendly message instead of displaying a technical error. Public users can view the expanded record details, while management actions are shown only when the user is logged in with the Admin role.

Section #6: (1 pt.) Admin Edit Project Record

100%

- Goal: show that an authorized admin can update approved record fields safely.
- Expected behavior:
 - The edit page validates the ID, preloads existing values, and validates submitted changes.
 - System-managed and API identity/source fields remain protected unless the project intentionally allows them.
 - Successful updates remain visible after refresh with friendly feedback.

Task #1 (1 pt.) – Admin Edit Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show deployed qa before-and-after evidence for one edit, including persistence after refresh.
- Show one friendly validation or invalid-ID message.
- Include focused code evidence for the editable-field allowlist, update logic, and role guard.



Deployed QA Admin edit form showing the existing Michos record values preloaded before changes.



Michos was updated successfully, with the new location saved on the deployed QA edit page.



Updated Michos location remains Bergenfield after refreshing the deployed QA edit page.



Invalid edit record ID redirects safely to the admin list with a friendly "Character record not found" message.

Part 2: URLs 100%

Details

- Paste a direct GitHub link for the edit page.
- Paste the anticipated production URL for one valid edit-page example.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/admin/edit_character.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/edit_character.php?id=4 🍏

Part 3: Response 100%

Details

- Explain which fields can change, why protected fields stay unchanged, and how updated values persist.

① Details

- Explain which fields can change, why protected fields stay unchanged, and how updated values persist.

Response

Admin users can update approved fields such as name, status, species, gender, origin, location, and image URL. System-managed fields such as the record ID, API ID, and record source are not editable, which protects the record's identity. Submitted values are validated and saved through a prepared update query, and the updated values remain visible after refreshing the page.

Supports **bold**, *italic*, [links](url), ``code``, etc.

376 chars

Section #7: (1 pt.) Admin Delete Or Deactivate Project Record

100%

- Goal: show that an authorized admin can safely remove or deactivate a record.
- Expected behavior:
 - Delete uses POST, validates the record ID, and follows one consistent hard-delete or soft-delete rule.
 - The destination repeats the role check and redirects with friendly feedback.

🔧 Task #1 (1 pt.) – Admin Delete Evidence

100%

🔧 Combo #1

🖼️ Part 1: Images 100%

① Details

- Show deployed qa evidence of one delete or deactivation and its friendly result message.
- Show the resulting list or database state.
- Include focused code evidence for POST handling, ID validation, the delete/deactivation query, and role guard.



Disposable manual record shown on the deployed QA Admin list before deletion.



Disposable manual record deleted successfully, with the Admin list reduced from five records to four.



Admin-only POST deletion with validated record ID, prepared hard-delete query, and friendly result handling.

🔗 Part 2: URLs 100%

① Details

- Paste a direct GitHub link for the delete handler.
- Identify the anticipated production delete-handler URL, without including a destructive request value.

① Details

- Paste a direct GitHub link for the delete handler.
- Identify the anticipated production delete-handler URL without including a destructive request value.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/admin/delete_character.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/delete_character.php 🍏

✍️ Part 3: Response 100%

① Details

- Explain the hard-delete or soft-delete choice and how the handler protects the intended record.

Response

This uses a hard delete, permanently removing the selected record from the Characters table. The handler requires the Admin role, accepts only POST requests, validates the record ID as a positive integer, and confirms that the record exists before running a prepared delete query. This protects against deleting the wrong record and redirects with a friendly success or error message.

Supports **bold**, *italic*, [links](url), ``code``, etc.

384 chars

Section #8: (1 pt.) Misc

100%

📁 Task #1 (0.33 pts.) – GitHub Details

100%

📁 Combo #1

🖼️ Part 1: Images 100%

① Details

- Screenshot the Commits tab of the Milestone 2 pull request.
- The screenshot should show meaningful API, database, create, list, details, edit, delete, and verification commits.



Milestone 2 pull request commits covering the database schema, shared helpers, character management workflows, and public charac

🔗 Part 2: URLs 100%

① Details

- Paste the pull request URL.
- The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

- Paste the pull request URL.

- The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/pull/24> 🍏

🖼️ Task #2 (0.33 pts.) – WakaTime Activity

100%

📄 Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name.
- Screenshot the file-level activity.
- The purpose is evidence of activity; the duration is not tied to the grade.
- The visual graphs are not required.



WakaTime project dashboard showing the overall coding time



WakaTime file-level activity showing time spent

☰ Task #3 (0.33 pts.) – Reflection

100%

📄 Details

- Answer each question with at least a few reasonable sentences.

✍️ Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I just simply learned how to connect API data to a database and manage both API and manual records.

Supports **bold**, *italic*, [links](url), ``code``, etc.

99 chars

✍️ Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

Finishing the page.

Response

Finishing the page.

Supports **bold**, *italic*, [links](url), `code`, etc.

19 chars

 Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

Debugging the files from Milestone 1. I had to keep going back and forth between multiple files to find and fix the problems. A skill issue on my part.

Supports **bold**, *italic*, [links](url), `code`, etc.

151 chars

End of Assignment - submission has been recorded.